Part 1

DŠA

- Prečo ?! — (i) Rýchlosť v čase; (ii) # operácií; (iii) Pamäť
- Začiatok / úvod do **teoretickej** informatiky
- analýza algo. ≠ benchmarking ← aplikovaná informatika
  - nezáleží na hardware – teoretická úroveň

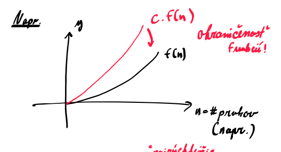
Zápis

- "Najhorší prípad": O-zápis
- "Najlepší prípad": Ω-zápis
- "Priemerný prípad": Θ-zápis "⇔" $O(g) \wedge \Omega(g)$

} Triedy funkcií · pôvod: mat (ohraničenosť, asymptoty + postupnosti)

Lekcia 1

Napr.



Napr. $f(x) = x^2 - x + 2 \Rightarrow O(x^2)$ ← "najrýchlejšie rastúci"

Príklad. Nech $f(n) = 3n^2 - 10n$

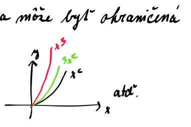
a) $3n^2 - 10n \Rightarrow O(n^2),\ n \geq 0$ — ohraničená $3n^2$

predtože $3n^2 - 10n \leq 3n^2$ vždy pre $n \geq 0$.

b) Taktiež $3n^2 - 10n \leq n^3 \Rightarrow O(n^3)$ ak $n \geq 10$. — ohraničená tiež n^3.

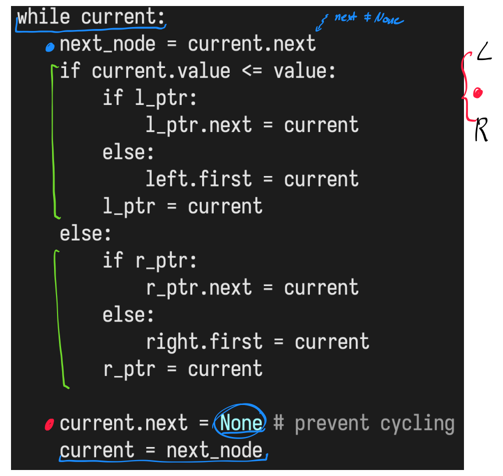
! pozor: vždy nás zaujíma "najnižšia" trieda, resp. ohraničenosť. ($3n^2 \leq n^3$)

Áno, funkcia môže byť ohraničená x rôznymi funkciami.



Lekcia 2

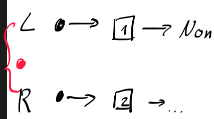
D.ú. 1 rámec:



```
while current:
    next_node = current.next
    if current.value <= value:
        if l_ptr:
            l_ptr.next = current
        else:
            left.first = current
        l_ptr = current
    else:
        if r_ptr:
            r_ptr.next = current
        else:
            right.first = current
        r_ptr = current

    current.next = None # prevent cycling
    current = next_node
```

next ≠ None



Cvičenia

$O(n) \wedge \Omega(n)$ — tento algo. $n^2, n^3, n^4, \dots, n!$

1.) $\Theta(n) \subseteq \Omega(n^2)$

$\{n\} \not\subseteq \{n^2, n^3, n^4, \dots, n!\}$

$\Omega(n) \not\subseteq \Omega(n^2) \quad \Omega(n^2) \subseteq \Omega(n)$

2.) $O(n) \subseteq \Omega(n)$

$\{1, \log n, \dots, n\} \not\subseteq \{n, n^2, n^3, \dots, n!\}$

Lekcia 3

Binary Search: $k = 2$ ← hľadané číslo

$[1, 2, 3, \boxed{4}, 4, 5, 6]$

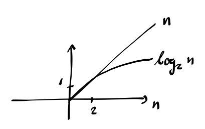
⇝ $[1, \boxed{2}, 3]$

⇝ ② ✓

Prečo log?

$[\ \dots \square \dots\ ]$ ⇝ $[\ \dots \square \dots\ ]$ ⇝ $[\ \square\ ]$ ⇝ … ⇝ $[x]$

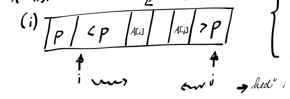
$\approx \log_{②} n \cong \log n$ (2 je nejaká konštanta) ← delíme na polovicu



Lekcia 4

Hoare Partition

A = list — in-place = O(1) (priestorová komplexnosť)



⇝ keď $i \geq j \Rightarrow$ swap p za A[j] a opakuj (i)!

$\Theta(n \log n)$; $\boxed{O(n^2)}$!

Príklad:

- hocijaký zoznam sorted (či už vzostupne alebo zostupne).

$\boxed{1}\ 2\ 3\ 4 \Rightarrow ①\ \boxed{2}\ 3\ 4 \Rightarrow ①\ ②\ \boxed{3}\ 4 \Rightarrow ①\ ②\ ③\ \boxed{4} \Rightarrow 1\ 2\ 3\ 4$ //

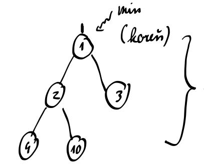
#4: n; #3: $n-1$; #2: $n-2$; … #1: 1 — Pre zoznam s n prvkami

$\Rightarrow n + (n-1) + (n-2) + \dots + 1$ to isté ako $1 + \dots + (n-1) + n$

$\Leftrightarrow \sum_{i=1}^{n} i = \frac{n}{2}(n+1) = \frac{n^2+n}{2} \boxed{\in O(n^2)}$

Heap

- HeapSort
- MinHeap / MaxHeap



} zápis ako list […].

- ! časová komplexnosť sa určuje zložitejšie.
  ↳ # rekurzívnych volaní
- niekedy viac intuitívne riešenie ako "iteratívne" algoritmy

D.ú. 4

- Prevod $(\dots)_{10} \to (\dots)_3$ (podobne ako pri dvojkovej)

ternary_sum(length: int, total: int) → list[int].

- ternary: {0,1,2}
  - rekurzia
  - $O(T(len, total) . len)$ ✓

T	1 … total
1	1
⋮	…

length } # možných kombinácií

bar & star problem + Inclusion-Exclusion (doplnok z MAT:)

Príklad: $len = 3 \wedge total = 5$

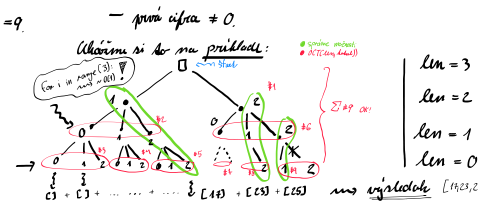
$T(3,5) = 3 \Rightarrow 3.3 = 9$.

$3^3 =$ 3 3 3 $= \sum 27$ ⇝ všetky možnosti

Algoritmus:

- prvá cifra ≠ 0.

Ukážme si to na príklade:



⇝ výsledok [17,23,25] //

- len = 3
- len = 2
- len = 1

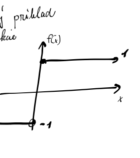
} rekurzia

- len = 0 ⇒ koniec

Lekcia 5

$$f(x) = \begin{cases} 1 & x \leq 0, \\ -1 & x > 0. \end{cases}$$

En: "Piecewise function"



4.1) Apple(n).

$$T_a(n) = \begin{cases} k_1 & n = 1 \vee n = 2, \\ T_a(n-1) + T_a(n-2) + n.k_2 & \text{inak.} \end{cases}$$

Banana(n).

$$T_b(n) = \begin{cases} k_1 & n = 1 \vee n = 2 \\ (*) & \text{inak.} \end{cases}$$

$(*)\ T_b(n-1) + \underset{i \in 2\dots n}{(n-1)}.\left(T_b(n-1) + k_2\right)$ "body" slučky.

$\Leftrightarrow T_b(n-1).1 + (n-1)\,T_b(n-1) + (n-1).k_2$

$\Leftrightarrow T_b(n-1)(\underbrace{1 + n-1}_{=n}) + (n-1).k_2 = n.T_b(n-1) + (n-1).k_2$ □

Orange(n)

$$T_c(n) = \begin{cases} k_1 & n = 1 \vee n = 2. \\ (*) & \text{inak.} \end{cases}$$

$(*) \Leftrightarrow T_c(n/2) + (n-1)(T_c(n-1))$ (?)

4.2) "Master Theorem"

↳ Divide and Conquer – $T(n/b)$

↳ nejaká funkcia $f(n) \in \Theta(n^c)$.

↳ Platí podobne pre O, Ω.

(N)

a)
$$T_a(n) = \begin{cases} 1 & n = 1, \\ T_a(n/2) + 1 & n > 1. \end{cases}$$

(+1 = f(n))

Teda: $a = 1, b = 2, c = 0 \Leftrightarrow n^0 = 1$.

$\Rightarrow 1 = 2^0 \Leftrightarrow 1 = 1. \Rightarrow \Theta(n^0.\log n) \Leftrightarrow \Theta(\log n)$. □

$\xrightarrow{/2} \xrightarrow{/2} \dots \xrightarrow{/2^k} 1 \} \leq \log_2 n \in \Theta(\log n)$.

c)
$$T_c = \begin{cases} 1 & n = 1, \\ 2T_c(n/2) + n & n > 1. \end{cases}$$

$a = 2, b = 2, c = 1 \Rightarrow 2 = 2^1$.

$\Rightarrow \Theta(n \log n)$. □

4.30) Hint: b) $T(n) = 2T(n-1) + n$

$T(n-1) = 2T(n-2) + (n-1)$

$\Leftrightarrow T(n) = 2(2T(n-2) + (n-1)) + n$

$②^2 T(n-2) + 2(n-1) + n$

Pokračuj s $T(n-2) = \dots \Leftrightarrow T(n) = 2.2^2(\dots) + \dots$ ②³ atď…

Vyvoď nejaké k.

Lekcia 6.

4.30b) Dokončenie

$$\text{Majme } T(n) = \begin{cases} 1 & n = 1, \\ 2T(n-1) + n & \text{inak.} \end{cases}$$

Rekurzívny vzťah:

$T(n-1) = 2T(n-2) + (n-1)$

$T(n-2) = 2T(n-3) + (n-2)$ — Zapíšeme ako

⋮ opakuj k-krát

$T(n-k) = 2T(n-k-1) + (n-k)$

Pomôž si príkladom :)

Napr.: $T(n) = 2(2T(n-2) + (n-1)) + n$

$2.2T(n-2)$ — $2^2 \dots 2^3 T(n-3)$ atď. $+ 2^1(n-1) + 2^0 n$ ($2^0 = 1$)

$2^2(n-2) + 2^3(n-3) \dots$ atď.

$\Longrightarrow T(n) = \underbrace{2^k (n-k)}_{\text{časť A}} + \underbrace{\sum_{i=0}^{k-1} (n-i)2^i}_{\text{časť B}}$. ⇝ Nejako použijeme "base case" T(1).

Kedy končí rekurzia? $n - k = 1 \Leftrightarrow k = n-1$

$\Rightarrow 2^{n-1} T(\underbrace{n - (n-1)}) + \sum_{i=0}^{n-2} 2^i (n-i)$

$\Leftrightarrow$ • $\underbrace{2^{n-1} T(1)}_{2^{n-1}} + \underbrace{\sum_{i=0}^{n-2} 2^i (n-i)}_{(*)}$ ($n - n + 1 = 1$) OK ✓

$(*)\ \sum_{i=0}^{n-2} 2^i (n-i) = ?$

$\Leftrightarrow 2^0 n + 2^1(n-1) + 2^2(n-2) + \dots + 2^{n-2}(\underbrace{n - (n-2)})$ ($n - n + 2 = 2$)

$\Leftrightarrow n + 2(n-1) + 2^2(n-2) + \dots + \underbrace{2^{n-2} . 2^1}_{2^{n-2+1} = 2^{n-1}}$

Potom spolu časť 1 a časť 2.

$\Leftrightarrow T(n) = 2^{n-1} + n + 2(n-1) + 2^2(n-2) + \dots + 2^{n-1}$

$\Leftrightarrow T(n) = 2.2^{n-1} + (\dots)$ zvyšok

$\Leftrightarrow T(n) = \textcircled{2^n} + \underbrace{n + 2(n-1) + 2^2(n-2) + \dots}_{\text{Očividne pomalšie rastúce ako } 2^n.}$ ← dominantný člen

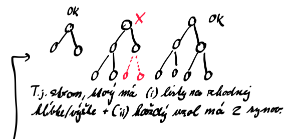
$\Rightarrow T(n) \in O(2^n)$. □ "dominant"

Lekcia 7

Kód:

Invariant/Podmienka

- listy majú rovnakú hĺbku/výšku,
  - 1 syn → ľavý syn
  - 2 synovia → ľavý syn koreňom perfektného stromu
- kľúč je min hodnôt synov

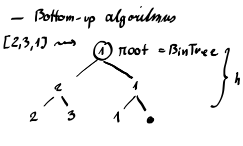


T.j. strom, ktorý má (i) listy na rovnakej hĺbke/výške + (ii) každý uzol má 2 synov.

1) leaves: list[int] → BinTree — Dôkaz ⟹

- Bottom-up algoritmus

[2,3,1] ⇝



} $h \approx \log_2 n$

Nech $T(n)$ je časová zložitosť, potom:

$T(n) = n + \lceil n/2 \rceil + \lceil n/4 \rceil + \dots + 1, \quad \lceil x \rceil < x + 1$

$\Leftrightarrow T(n) < n + (n/2 + 1) + (n/4 + 1) + \dots + 1$

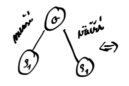
$\Leftrightarrow T(n) < n(1 + \underbrace{\tfrac{1}{2} + \tfrac{1}{4} + \dots}_{\approx 1}) + (\underbrace{1 + 1 + \dots}_{\log_2 n} + 1)$

$\Leftrightarrow T(n) < n.2 + \log_2 n + 1$

$\Rightarrow T(n) \leq O(2n + \log_2 n) \leq O(2n + \log n) \leq O(2n) \leq O(n)$ □

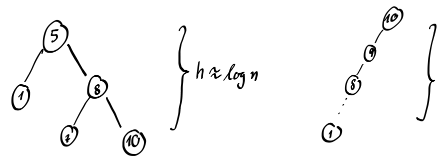
Stromy

↳ ADS: abs. dátová štruktúra:



$\Leftrightarrow s_1 \leq \sigma \leq s_2$

Príklad:



⇒ Balanced Trees

Red-Black Tree (jeden z mnohých typov)

"Invariant"/Podmienka

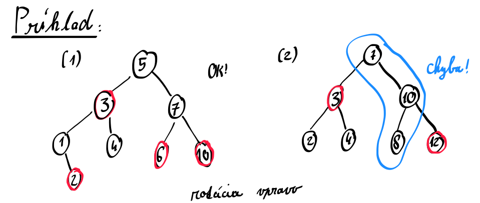
- koreň = čierny
- každá "cesta" má rovnaký # čiernych
- otec červený ⇒ syn čierny

Operácie

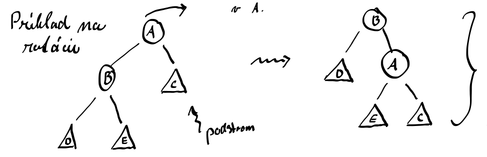
- Search
- Insert
- Remove

} $O(\log n)$ — treba dodržať podmienku + rotácie $O(1)$.

Príklad:



Príklad na rotáciu



} Dôležité pre každý typ balansovaného stromu :)

Haldy

↳ ADS

1) Chceme $O(n)$, n = heap.array.

"Naivný algo": […] (n) d-krát $\leq O(n.d)$ X

"Lepší algo" ⟹ Chceme otcov, potom pozrieme synov.

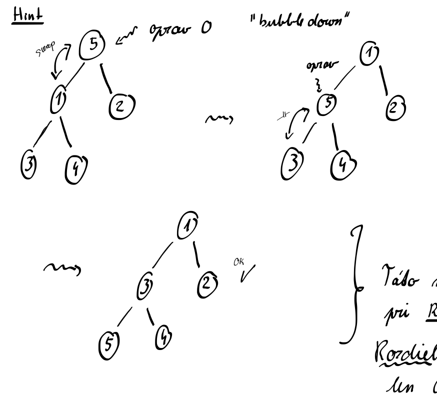
Hint: Počet otcov $= \underbrace{\lfloor (n-1)/d \rfloor}_{\text{trik :)}}$.

- Prvý for-loop: $\lfloor (n-1)/d \rfloor \leq O((n-1)/d) \leq O(n/d)$. ⇝ // v Py :)
- Druhý for-loop: $O(d)$

$\Leftrightarrow O(n/d).O(d)$

$\Leftrightarrow O(n/d.d) \leq O(n)$ □

2) Hint



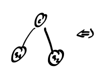
} Táto metóda sa používa pri Remove pri MinHeap. Rozdiel: inde nemusí byť len σ!

Bonus

Halda: → Min/Max

- Insert – "Bubble up" ↑
- Delete – "Bubble down" ↓

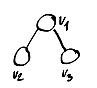
Podmienka:



$\Leftrightarrow \sigma \leq s_1 \wedge \sigma \leq s_2$!

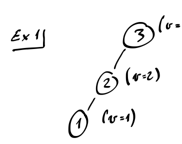
Lekcia 8.

- k-vyvážený strom: $\in [1/2, 1]$



$\Leftrightarrow v_2 \leq k.v_1 \wedge v_3 \leq k.v_1$.

Ex 1



$2 \leq k.3 \Leftrightarrow k = \frac{2}{3}$ — $2 \leq 1/3 . 3 \Leftrightarrow 2 \leq 1$ x

$1 \leq k.2 ⇝ k = \frac{2}{3}$

$1 \leq \frac{2}{3}.2$

$1 \leq \frac{4}{3}$ ✓

Part 2

Test 1

1) 'zip' ⇒ [(1, "Jirik"), (0, "rs"), (0, "He"), (1, "Boss!")] — take

→ "with" take

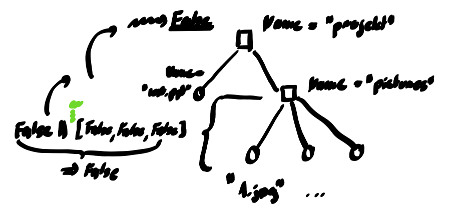
⋮

4) • type Name = [Char] ("typový konštruktor") (typový alias)

• data FileNameSystem = File Name | Dir Name [FileNameSystem] ("hodnotový konštruktor") — rekurzívna časť; Napr. z matiky :) $a_n = a_{n-1}$

File Name ⇝ ○; Dir Name [FileNameSystem] ⇝ □; "V"

pre súbor vyzerá ⇝ a = File "test.hs" ○



"trees – stromy" DŠA! :

Úlohy:

a) Dir "projekt" [File "1.jpg"]

b) FileNameSystem

Test 2.

1) filter (flip elem "M!o") "Master of orion 2 nemá taký kravský level!"

$\lambda x \to$ ~~flip~~ elem "M!o" x — flip elem "M!o" x ⇒ elem x "M!o" ✓

Príklad: $\lambda x \to$ elem "M!o" x — !! nemení

elem :: Eq a ⇒ a → [a] → Bool

Výstup: nový reťazec ⇝ "Mooo!"

2) [1,2,3,4] } tail [2,3,4] ⟩ +zip [(1,2), (2,3), (3,4)]

3.) head :: [a] → a ("a" je špecifický k funkcii)

tail :: [a] → [a]

(.) :: (b → c) → (a → b) → a → c ⇔ g ∘ f ⇝ g(f(x)) !

$\lambda x \to$ (tail . head) x ⟺ { (.) tail head / tail . head ⇝ Príklad (+) 1 2 / 1 + 2

Aby toto platilo výstup head = vstupu tail-u

Riešenie: uvažujem a ~ [a]. ⇒ g∘f :: [[a]] → [a].

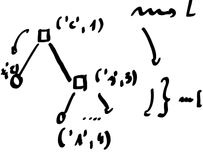
4) type Code = (Char, Int)

data Allowed = Down Code | Section Code [Allowed]

a) Section ('A', 123) [Down ('B', 234)]

b) Allowed

c)



Test 3.

3.) 'a' :: Char

(:) :: a → [a] → [a]

$\lambda x\ y \to$ 'a' : (x : y) :: Char → [Char] → [Char] — jediný konkrétny typ

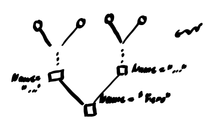
1) 'a' : xs :: Char → [Char]

2) (x:y) :: a → [a]

} spolu

4.) data Person = Unknown | WithParents Name Person Person

Unknown ⇝ ○; WithParents … ⇝ □



Test 4.

3) map :: (a → b) → [a] → [b] (x)

isUpper :: Char → Bool

head :: [a] → a

map ($\lambda x \to$ isUpper (head x)) :: [[Char]] → [Bool] //

1) head x ⇝ x :: [a]
2) isUpper ("a") ⇒ a ~ Char → Bool
3) (1), (2) ⇒ x :: [Char]
4) (x) ⇒ [a] ~ [[Char]] = [String]
5) $\lambda x \to$ isUpper (head x) :: Bool
6) Výstup: [Bool]

4) type Voltage = Int

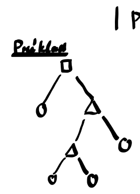
type Current = Int

data EPower = Bat Voltage Current ○

| Serial EPower EPower } □

| Parallel EPower EPower } Δ

Príklad



a) Bat 1 1 ⇒ napätie 2, prúd 2.

Parallel { prúd ⊕ (Current); napätie "max" (Voltage) }

Serial { prúd "max"; napätie ⊕ }

b) EPower

c) measureV :: EPower → Voltage Parallel (Serial (Bat 1 1) (Bat 1 1)) (Bat 1 1)

⇒ Bat 2 1

↳ Parallel (Bat 2 1) (Bat 1 1)

⇒ Bat (2 2) //

Typy

→ Dané: { flip :: (a → b → c) → b → a → c; replicate :: Int → a → [a] }

$\boxed{\lambda x \to \text{flip replicate} :: (?)}$

i) flip replicate :: a → Int → [a]

ii) a → (b → Int → [b]) (a nevyplývajú b)

iii) a → b → Int → [b] (funkcia (x) má vlastné vstupné hodnoty) //

2) (:) :: a → [a] → [a]

True :: Bool

(++) :: [a] → [a] → [a]

$\lambda x\ y \to$ True : (x ++ y) :: [Bool] → [Bool] → [Bool] //

typ Maybe:

data Maybe = Nothing | Just (x)

Príklad: f(x,y) = x / y :: Int → Int → Double (Použi Maybe)

! delenie 0 nie je Ok!

f _ 0 = Nothing

f x y = Just (x / y)

1.) fst :: (a, b) → a

snd :: (a, b) → b

(.) :: (b → c) → (a → b) → a → c

$\lambda x \to$ (fst . snd) x :: (a, (c, b)) → c

i) b ~ (a, b) Teda snd (a, (c, b)) → (c, b)

ii) fst (c, b) → c.

3.) Maybe :: x → x

fromMaybe :: a → Maybe a → a

map :: (a → b) → [a] → [b]

$\lambda x \to$ map (fromMaybe x) [Just 3, Nothing, Just 5] :: Int → [Int] //

4.) map ($\lambda x \to$ null (filter last x)) :: [[[Bool]]] → [Bool] //